Repo z przykładami

README MIT license

Bulletproof frontend testing

This repository supports a series of talks and workshops focused on testing frontend applications the bulletproof way. Its aim is to provide testing suites, cases and patterns as well as rich explanations of their rationales.

It contains an example React SPA with a thin layer of supporting backend. Testing suites cover integration, functionality, accessibility, visual regression and so on.

The talks were performed during:

- Voxxed Days Crete conference talk (26-27.09.2025 - Heraklion, Crete Island, Greece)
- React Alicante workshop (02-04.10.2025 - Alicante, Spain)

https://github.com/LukaszNowakPL/bulletproof-frontend-testing

Wartościowe i kuloodporne testy frontendów

Łukasz Nowak, TestWarez2025

Wisła, 05 Listopad 2025

Poznajmy się



Łukasz Nowak

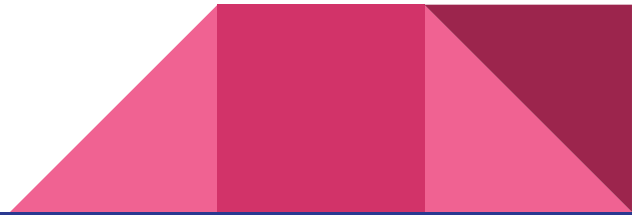
Senior frontend dev

Łukasz Nowak

LukaszNowakPL

Zakładana agenda

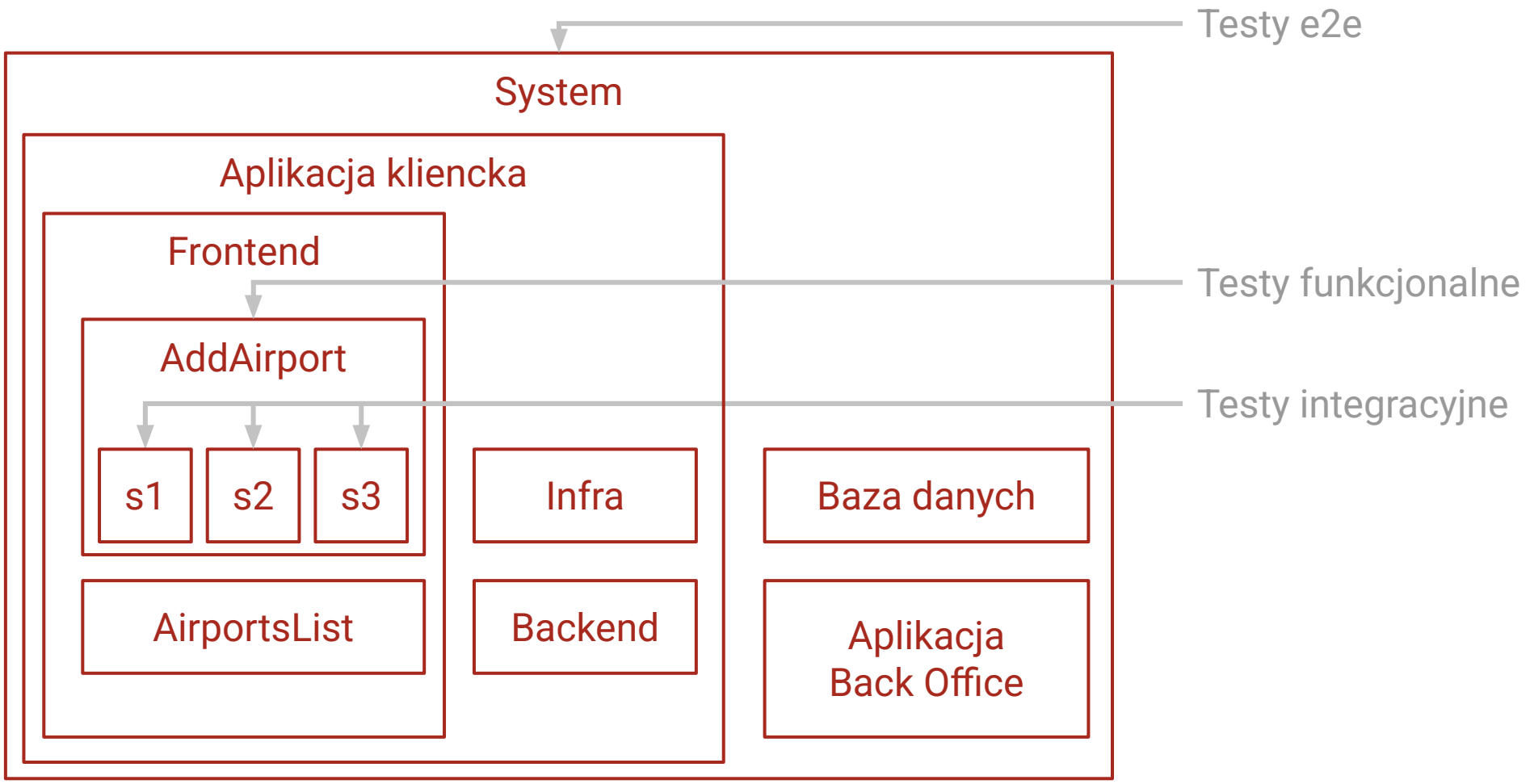
- Testy funkcjonalne i e2e
- Doktoryzacja w mockowaniu api
- Testy accessibility
- Testy wizualnej regresji
- Testy integracyjne dla developerów



Testy w Playwright (funkcjonalne & e2e)

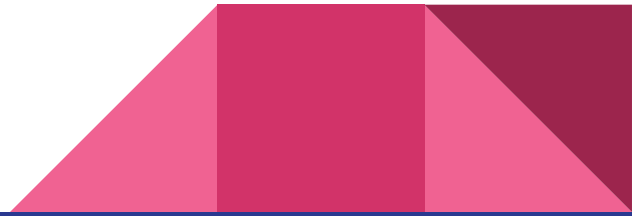
Idea testów funkcjonalnych

- Testy integracyjne (dev)
  - Testy na surowym kodzie części aplikacji (komponent, widok lub jego część)
- Testy funkcjonalne (dev lub qa)
  - Testy na “binarce” aplikacji, wykonywane na lokalnej maszynie
  - Potwierdzają działanie funkcjonalności na froncie
    - One pager - bardzo podobne do testów integracyjnych
    - Multi stepper - tutaj ujawniają swoją moc
  - Połączenie z backendem jest zamockowane
- Testy e2e (qa)
  - Testy aplikacji zdeployowanej na dedykowane środowisko
  - Potwierdzają cross-projektowe działanie systemu - backend i frontend, być może kilka aplikacji w ramach jednego scenariusza


Testy e2e
System
Aplikacja kliencka
Frontend
Testy funkcjonalne
AddAirport
Testy integracyjne
s1
s2
s3
Infra
Baza danych
AirportsList
Backend
Aplikacja
Back Office

Narzędzie do testów funkcjonalnych

- Playwright czy Cypress?
  - Cypress używa specyficznej składni (łańcuch zdarzeń)
  - Playwright jest czystym JavaScript
  - Playwright bardzo przypomina Jest/Vitest
    (wspólny język z developerami, fajne przy shift left)
  - Mocno tkwi w Open Source, bez rozwiązań wymagających subskrypcji
- Moc Playwright tkwi w dodatkach
  - Składnia zapożyczona z Testing Library (getByRole)
  - Tworzenie snapshotów (np. wizualna regresja)
  - Integracja z zewnętrznymi serwisami (np. Lighouse lub Axe)
  - Nawigacja klawiszami



Do czego wykorzystuję Playwright

- Testy funkcjonalne
  - Funkcjonalne journey
  - Journey z nawigacją klawiaturową (a11y)
- Testy snapshotów
  - Wizualna regresja
  - Statyczna analiza pod a11y

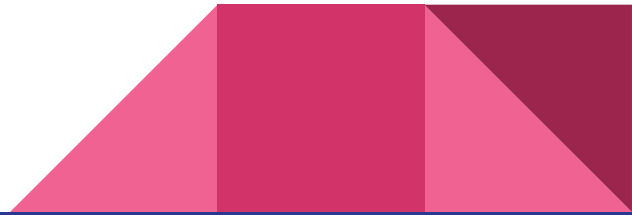


Setup środowiska testowego

Przykład testu funkcjonalnego

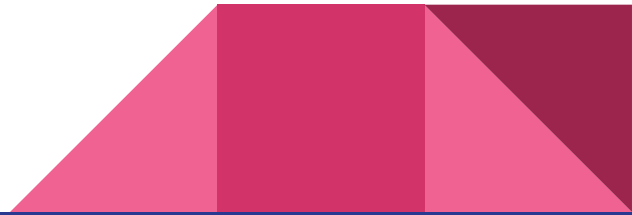
Dobre praktyki

- Happy i Negative paths
- Notacja Gherkina
- Page object pattern
  - Funkcje obsługujące konkretne akcje
- Definiowanie danych bazowych i reużywalność w scenariuszu
  - Jasne nazewnictwo danych
  - Scenariusz jest właścicielem danych, bo wie z czego musi korzystać
  - Type safety używanych danych
- Ewaluacja komunikacji z api

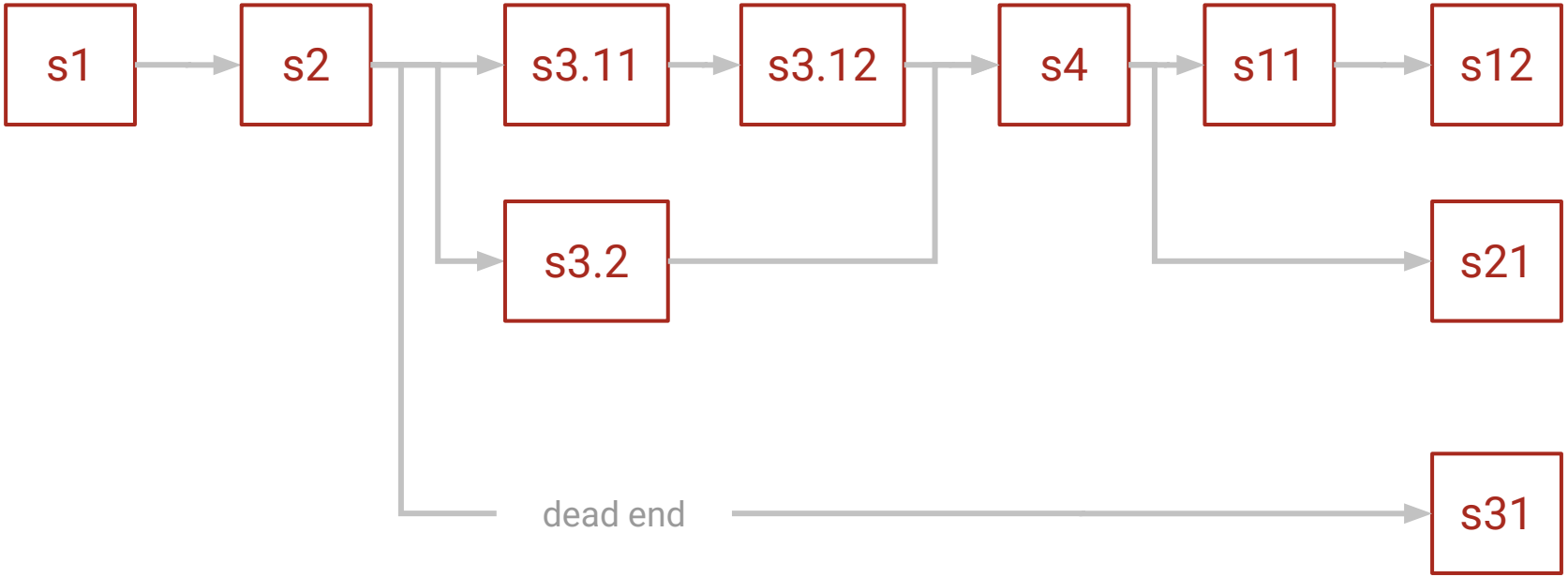


Tips & tricks na poziomie pro+

- Używanie data-test-id to zły pattern
- Dbanie o czystą konsolę
- Brak przesady z ilością danych w mockach api
- Upewnienie się że strona jest gotowa do testu/akcji
  - Wykonywanie akcji stopniowo, gdy jest ona w stanie jałowym
- Stosowanie wyrażeń regularnych zamiast stringów
- Generator scenariuszy dla multistepperów



Funkcjonalności wielokrokowe



Mockowanie komunikacji z api

Przykładowe repozytorium
(dedykowane mockowaniu api)

README MIT license

Advanced api mocking patterns

This repository supports `Advanced patterns of api mocking in tests` talk. Its aim is to explain several most common mistakes on tests of applications/components triggering api endpoint calls. It also contains example of factory pattern and data randomization to better reflect unpredictable way users use the application.

This talk was performed during:

- Online track of Warszawskie Dni Informatyki conference (04.04.2025)

Project structure

The project contains React frontend application and supporting backend stub service based on `json-server` component.

Test examples supporting each chapter of the talk are groupped on /frontend/test directory. To cover both - developer and Tester/QA/SDET - approach, each chapter contains Playwright functional examples as well as Vitest integration examples.

https://github.com/LukaszNowakPL/advanced-api-mocking-patterns

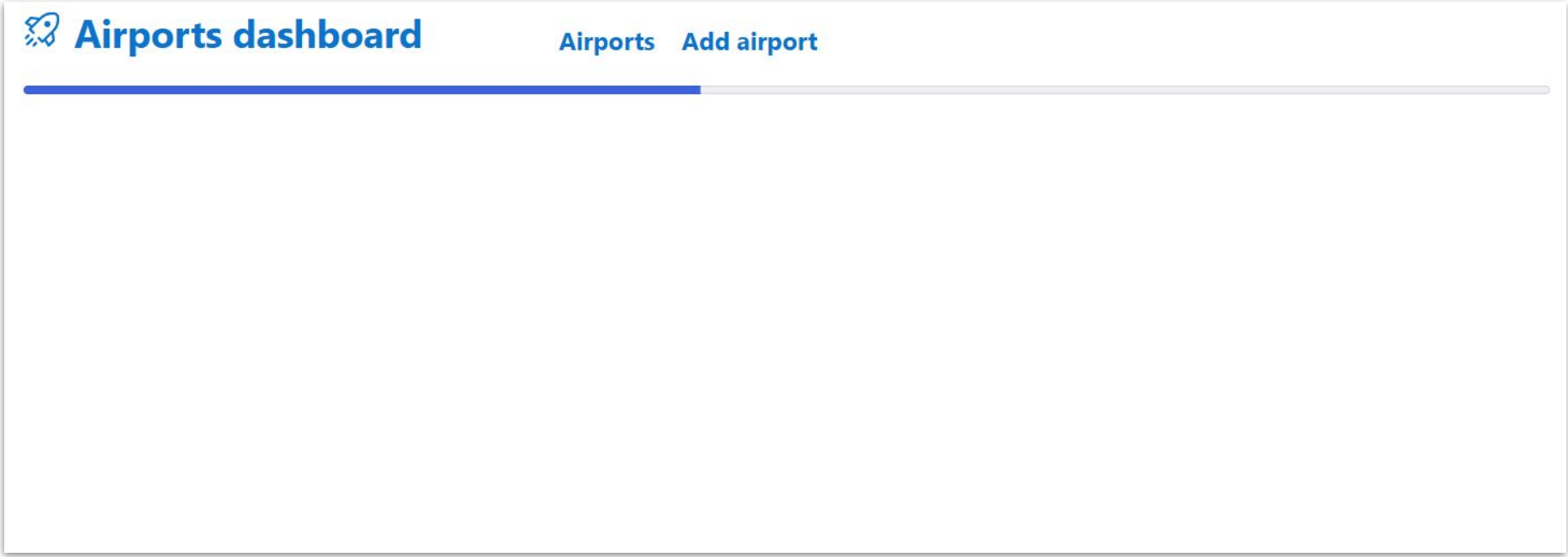
Brak mockowania komunikacji z api

Brak mockowania komunikacji z api

- Nie mockujemy, bo nie wspiera bezpośrednio naszego testu

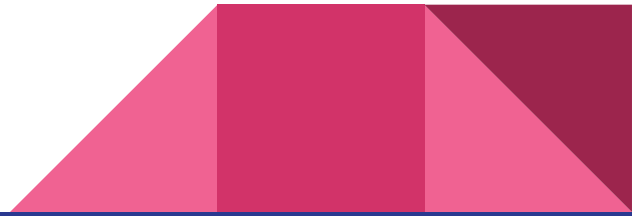


Testowanie na obecność loadera



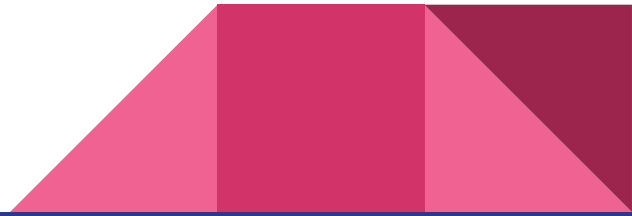
Brak mockowania komunikacji z api

- Nie mockujemy, bo nie wspiera bezpośrednio naszego testu
- Komunikacja jest asynchroniczna, jej wynik może być dostępny dopiero po zakończeniu testu
- Powoduje losowo wyglądające błędy przy rozbudowanych suitach testowych



Jak to naprawić?

- Nie używać domorosłych patternów
- Dodać tooling do mockowania komunikacji z api
- Zamockować połączenia wykonywane w trakcie testu



Mockowanie w MSW

```ts
// api-handlers/countries.ts
import {http, HttpResponse} from 'msw';
import {CountriesDto} from '../../../src/api/rest/countries.dto';

export const countriesHandler = (responseData: CountriesDto, status = 200) =>
   http.get(`*/api/countries`, () => HttpResponse.json(responseData, {status}));
```

```ts
// AirportView.test.tsx
server.use(
   countriesHandler([countryData]),
   regionsHandler([]),
);
```

Mockowanie w Playwright

```
test.beforeEach(async ({page}) => {
   mockiavelli = await Mockiavelli.setup(page);
});

test('Airport addition journey', async ({page}) => {
   // And mocks of api calls triggered during the test
   const postAirportMock =
      mockiavelli.mockPOST('http://localhost:4000/api/airports', {
         status: 200,
         body: {},
   });
   // ...
})
```

Jak to naprawić?

- Nie używać domorosłych patternów
- Dodać tooling do mockowania komunikacji z api
- Zamockować połączenia wykonywane w trakcie testu
- Stworzyć listener informujący o brakującym mocku



Ustawienie listenera w MSW

```
beforeAll(() =>
    server.listen({
        onUnhandledRequest ({method, url}) {
            console.error('Found an unhandled %s request to %s' , method,
url);
        },
    }),
);

// Found an unhandled GET request to http://localhost:4000/api/airports/1
```

Mockowanie nadmiarowe

Mockowanie nadmiarowe

- Błąd mylnie rozumianego patternu DRY
- Najczęstsze przypadki
  - w beforeEach() grupy testów
  - w setupie środowiska do mockowania
  - stosowanie wildcards, np. dla /airports/*

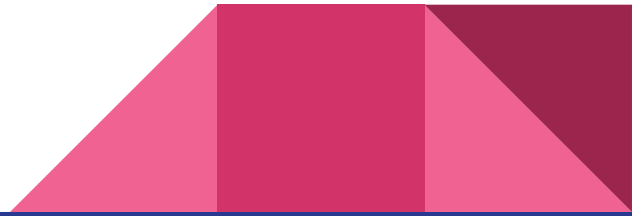


Hardcodowanie mocków na setupie MSW

```
export const server = setupServer(
   http.get(`*/api/countries`, () => HttpResponse.json(countriesList)),
   http.get(`*/api/airports`, () => HttpResponse.json(airportsList)),
   http.get(`*/api/airports/*`, () => HttpResponse.json(airport))
   // ...
);
```

Jak to naprawić?

- Mockować każde połączenia per test



Mockowanie połączenia w teście

```
// Given airport id
const airportId = 1;

// And Airport data returned by api call
const airportData: AirportDto = {
   id: airportId,
   // ...
};

// And GET /airport/{airportId} handler
server.use(airportHandler(airportId, airportData));
```

Mockowanie restrykcyjne

Mockowanie restrykcyjne

- Mockowanie tylko połączeń realnie wykonanych w teście
- Brak obsługi pozostałych połączeń - będzie info w konsoli
- Konieczność potwierdzenia:
  - Parametrów ścieżki
  - Danych w body
  - Pozostałych części istotnych dla testu - np. Search params, Headers, Cookies
- Pattern najdokładniej testujący integrację z backendem



Mockowanie restrykcyjne w MSW

```
export const addAirportHandler = (formData: AirportModel, status = 200) => {
    return http.post('*/api/airports', async ({request}) => {
        if (await isModelCorrect(formData, request)) {
            return HttpResponse.json({}, {status});
        }
    });
};
```

Porównanie obiektów

```
const isModelCorrect = async (formData: any, request) => {
    const requestBody = await request.clone().text();
    const stringifiedFormData = JSON.stringify(formData);

    if (requestBody === stringifiedFormData) {
        return true;
    } else {
        console.error('Body data does not match');
        return false;
    }
};
```

Restrictive mocking in Mockiavelli

```
// And new airport data
const airport: AirportModel = {
   name: 'test airport name',
   iata: 'TES',
   country_id: countries[0].id,
   regions: [regionToSelect.id],
   vaccination_notes: 'test vaccination notes',
};
//...

// Then POST api call is resolved with expected body
const postAirportRequest = await postAirportMock.waitForRequest();
expect(postAirportRequest.body).toEqual(airport);
```

Stosowanie wzorca fabryka

Powtarzalne tworzenie danych zwracanych przez api

```
// Given airport id
const airportId = 1;

// And Airport data returned by api call
const airportData: AirportDto = {
   id: airportId,
   // ...
};

// And GET /airport/{airportId} handler
server.use(airportHandler(airportId, airportData));
```

Pomysł: Obiekt bazowy do budowy mocków

```
// data-mocks.ts
const baseAirport: AirportModel = {
    name: 'test airport',
    iata: 'TES',
    country id: 1,
    regions: []
}

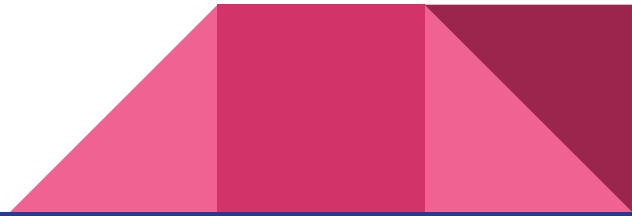
// Component.test.tsx
const spanishAirport = {
    ...baseAirport,
    country_id: 2
}
```

Baza zbyt skomplikowana dla spread operator

```
const complicatedDto = {
    client: {
        name: 'test name',
        surname: 'test surname'
    },
    contact: {/* ... */},
    address: {
        permanent: {/* ... */},
        temporary: {/* ... */}
    }
}
```

Stosowanie wzorca fabryka

- Obiekt bazowy do budowy mocków
- Spread operator vs. dedykowana biblioteka



Przykład użycia Factory.ts

```ts
// factories/countries.ts
import * as Factory from 'factory.ts';

export const countryFactory = Factory.Sync.makeFactory<CountryDto>({
   id: 1,
   name: 'test country name',
   is_in_schengen: true,
});
```

```ts
// AirportView.test.tsx
// And country mock
const country = countryFactory.build({name: 'different country name' });

// And mocks of api calls triggered during the test
server.use(countriesHandler([country]) /* ... */);
```

Stosowanie wzorca fabryka

- Obiekt bazowy do budowy mocków
- Spread operator vs. dedykowana biblioteka
- Uwaga na pułapkę: Nigdy nie ufaj obiektowi bazowemu, zawsze nadpisuj to co chcesz później testować.



Błędne poleganie na obiekcie bazowym

```
// factories/countries.ts
import * as Factory from 'factory.ts';

export const countryFactory = Factory.Sync.makeFactory<CountryDto>({
   id: 1,
   name: 'test country name',
   is_in_schengen: true,
});
```

```
// AirportView.test.tsx
// And schengen country
const schengenLikeCountry = countryFactory.build({name: 'other country name'});

// And schengen-related test checks
// ...
```

Nadpisanie wszystkich testowanych elementów

```
export const countryFactory = Factory.Sync.makeFactory<CountryDto>({
    id: 1,
    name: 'test country name',
    is_in_schengen: true,
});
```

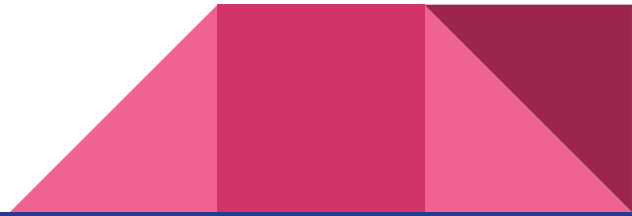
```
// And schengen country
const schengenCountry = countryFactory.build({
    name: 'test country name',
    is_in_schengen: true
});

// And schengen-related test checks
```

Użycie losowości do tworzenia mocków

Użycie losowości do tworzenia mocków

- Założenie: Cały obiekt bazowy jest losowy
- Efekt: Konieczność nadpisania danych używanych w teście
- Wynik: Obiekt bazowy zapewnia tylko zgodność z Dto/Modelem
  Odpowiedzialność za treść danych leży w warstwie testu
- Konieczność: Losowanie na poziomie enterprise



Przykład losowości danych z Faker library

```
import * as Factory from 'factory.ts';
import {faker} from '@faker-js/faker';

export const airportFactory = Factory.Sync.makeFactory<AirportDto>({
    id: faker.number.int(),
    name: faker.airline.airport().name,
    iata: faker.airline.airport().iataCode,
    country_id: faker.number.int(),
    regions: faker.helpers.multiple(
        () => faker.number.int(), {count: faker.number.int(5)}
    ),
});
```

Nadpisanie danych używanych w teście

```
// And country data
const country = {
   id: 1, is_in_schengen: false,
   is_passport_required: true, is_visa_required: false
}
const nonSchengenCountryData  = countryFactory.build(country)

// And airport data
const airport = {id: 99, name: 'test airport name', country: country.id}
const airportData = airportFactory.build(airport)

// And mocks of api calls triggered during the test
await countriesMock(page, [nonSchengenCountryData])
await airportMock(page, airport.id, airportData)

// And test for non-schengen airport ...
```

Co z tego mamy?

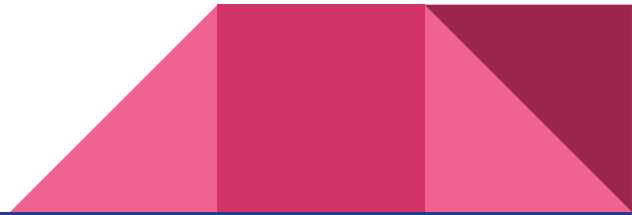
- Wady
  - Komplikuje proces tworzenia mocków
  - Komplikuje debugowanie testów
  - Wymaga sporego doświadczenia w pisaniu miarodajnych testów
- Zalety
  - Najlepiej oddaje nieprzewidywalność akcji użytkowników
  - Przypadkowo może znaleźć błąd w aplikacji



Mockowanie api w praktyce

Kroki do zapamiętania

- Ręcznie tworzyć dane api używane w teście
  - Test jest właścicielem kształtu danych z api
- Nadpisywać obiekty wypełnione danymi losowymi (Factory + Faker)
- Przekazywać dane do handlerów obsługujących komunikację a api
- Handlować/mockować tylko oczekiwane połączenia
  - Z dokładnością do url, body i in.



Testy accessibility

Aktualny stan a11y w web developmencie

- Temat często poruszany na konferencjach / meetupach
- 28.06.2025 - weszła w życie regulacja European Accessibility Act wymagająca spełnienie zaleceń WCAG 2.1 na poziomie AA
- Większość serwisów nie pokrywa w całości wymogów dostępności
- Bo każdy z nas ma dobre wymówki

Byłem wśród większości.
Do pewnego momentu.



Auditing Design Systems for Accessibility by Anna E. Cook

https://www.youtube.com/watch?v=L0PRzlyP1Zc

“Accessibility is not about being perfect. It’s about trying.”

Anna E. Cook, Wey Wey Web 2023

“It’s best to try a11y testing on new applications. You start small and grow as the project grows.”

Źródło nieznane

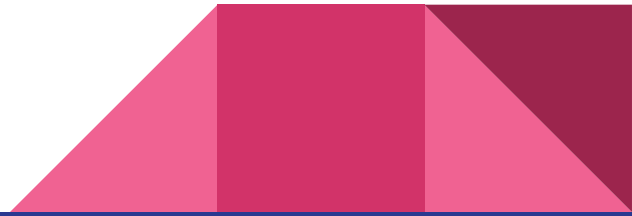
Czym tak naprawdę jest to accessibility?

A11y jest niezbędne dla klientów doświadczających

- Problemów ze wzrokiem lub słuchem
- Zaburzeń poznawczych i rozumienia treści
- Problemów ruchowych
- Zaburzeń mowy
- Ograniczeń neurologicznych

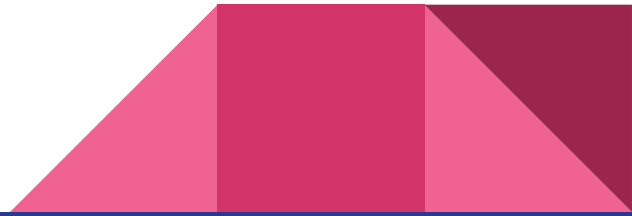
Najczęstsze wyzwania a11y w webdev

- Mały kontrast kolorów - do pokrycia przez design
- Skomplikowana treść lub struktura usługi - do pokrycia przez UX
- Niedostateczna identyfikacja elementów na stronie - obszar developmentu
- Problematyczna interakcja z elementami - to też



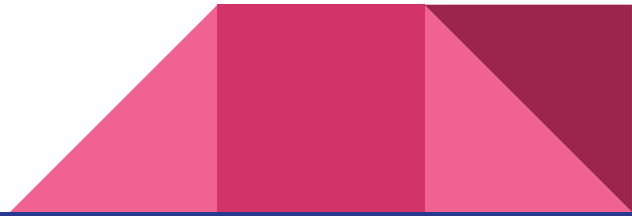
Niedostateczna identyfikacja elementów na stronie

- Wiele sposobów na zaprezentowanie treści strony
  - Wizualna prezentacja (ekran)
  - Odczyt audio (czytnik ekranu)
  - Zaawansowana technologia wspierającą (np. czytnik braille-a)
- Brak/słaby opis elementów typu obrazki, linki, przyciski i in.
- Wszystko rozbija się o jakość produkowanego HTML



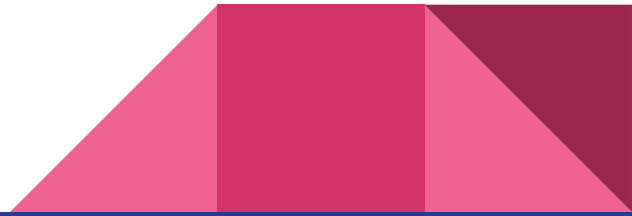
Problematyczna interakcja z elementami

- Używanie elementów strony to:
  - Wybranie elementu
    - Przez wyszukanie na stronie (np. getByRole(‘button’, {name: ‘Send’})
    - Przez listę elementów na stronie (nawigacja z użyciem przycisku Tab)
  - Wykonanie akcji dedykowanej dla elementu (wpisanie danych, klik, itp.)
- Wszystko rozbija się o semantykę produkowanego HTML



Punkt startowy do testowania

- Dostępność zapewnia semantyczny i dobrze opisany HTML
- Nie trzeba testować na każdym typie assistive technology
- Testowanie na poziomie produkcyjnego bundle
- Playwright (chociaż inne narzędzia też powinny dać radę)



Airports dashboard

Airports Add airport

Add airport

Name

Test airport name

IATA Code

TES

Country

Spain

Regions

island

- [ ] Aegean Islands
- [ ] Corfu Island (Kerkyra)
- [ ] Ha Long Island
- [ ] Rhodes Island
- [x] Balearic Islands
- [ ] Crete Island
- [ ] Ionian Islands
- [ ] Tenerife Island
- [ ] Canary Islands
- [ ] Dodecanese Islands
- [x] Maiorca Island

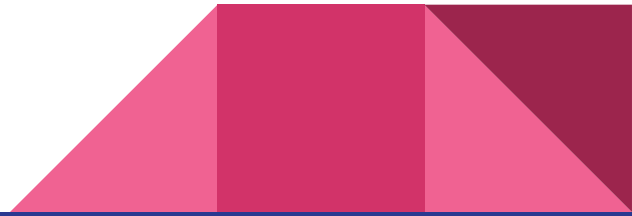
Vaccination Notes

Submit

W jaki sposób automatycznie testować a11y?

User journey z klawiaturową nawigacją

- To nic więcej jak kolejny scenariusz funkcjonalny
- Plan minimum: pokryć najprostszą happy path

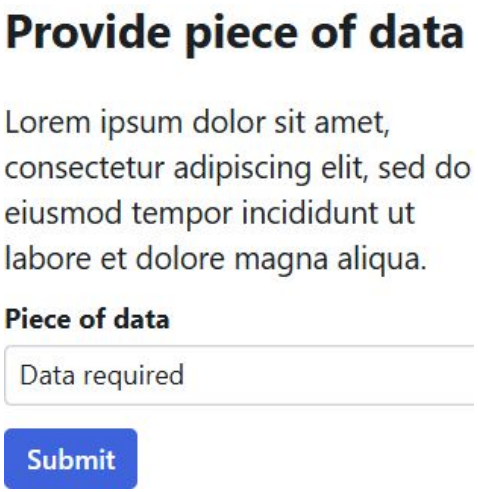


Audyt finalnego HTML

Wiele twarzy jednego widoku







Plan scenariusza



Wycinek z testu

```
// Then I wait until the page is ready
await addAirportPage.assertReady();

// When I perform snapshot analysis for initial view
await addAirportPage.performSnapshotAnalysis('Initial view', {takeLighthouseSnapshot});

// And I fulfill the form with correct data
await addAirportPage.fulfillForm(airport);

// Then I can see the form data is valid and Submit button is enabled
await addAirportPage.assertSubmitEnabled();

// When I perform snapshot analysis for provided data
await addAirportPage.performSnapshotAnalysis('Form filled', {takeLighthouseSnapshot});
// ...
```

Proces audytowania

```
public async assertSnapshots(snapshotTitle: string, config?: AssertSnapshotConfig) {
    await Promise.all([
        this.assertAxeChecks(snapshotTitle, Boolean(config?.skipAxeViolationChecks)),
        this.performAriaSnapshotComparison(snapshotTitle),
        config?.takeLighthouseSnapshot !== undefined
          && this.assertLighthouseSnapshot (snapshotTitle,
config.takeLighthouseSnapshot ),
    ]);
}
```

Audyt Lighthouse

- playwright-lighthouse - zintegrowana automatyzacja
- Fajnie craftowy audyt accessibility:
  - Best practices, WCAG, semantyka HTML
  - Analiza wizualna (kontrasty kolorów, odległości itp.)
- Prosty w zrozumieniu i pomocny raport końcowy



https://www.npmjs.com/package/playwright-lighthouse

Zapięcie Lighthouse Flow

```
// And Lighthouse flow
const flow = await LighthousePlaywrightFlow
     .startFlow('Airport addition journey', page);

// And take snapshot helper to be passed to each page
const takeLighthouseSnapshot: TakeLighthouseSnapshot = async (name) =>
   await flow.snapshot(name,
         {onlyCategories: Object.keys(lighthouseThresholds), formFactor: 'mobile'});

// ...
// When I perform snapshot analysis for initial view
await addAirportPage.performSnapshotAnalysis('Initial view',
     {takeLighthouseSnapshot, skipAxeViolationChecks: true});

// ...
// Then I generate snapshot report
await flow.generateReports(testInfo, 'lighthouse-test-add-airport-happy-path-flow.html',
lighthouseThresholds);
```

Tworzenie raportu z Lighthouse flow

```
const res = await this.flow.createFlowResult();

// Attach Lighthouse report to Playwright report
await testInfo.attach(filename, {body: reports.toString()});

const lighthouseThresholds = {'best-practices': 100, accessibility: 90, seo: 50};

const categories = Object.keys(lighthouseThresholds)
    as ('accessibility' | 'best-practices' | 'seo')[];
let thresholdFailures: string[] = [];

res.steps.forEach((step) => {/* ... */});

expect(thresholdFailures).toStrictEqual([]);
```

Axe accessibility testing engine

- Statyczna analiza HTML u rynkowego lidera
- Raporty z testów - słodko-gorzki format
- Możliwość zawężania skanowania
  do wybranych wytycznych (np. WCAG 2.1 AA)



https://playwright.dev/docs/accessibility-testing

Wywołanie statycznej analizy HTML w Axe

```
private async assertAxeChecks(snapshotTitle: string, skipViolationChecks: boolean) {
    const axeChecks = await new AxeBuilder({page: this.page})
        .withTags(['wcag2a', 'wcag2aa', 'wcag21a', 'wcag21aa', 'wcag22aa'])
        .analyze();

    await this.testInfo.attach(this.composeA11yReportName(snapshotTitle), {
        body: JSON.stringify(axeChecks, null, 2),
        contentType: 'application/json',
    });

    if (!skipViolationChecks) {
        expect(axeChecks.violations).toEqual([]);
    }
}
```

Aria snapshot

- Idea drzewa dostępności
- Coś jak DOM dedykowany dla a11y
- Funkcjonalność Aria snapshots w Playwright
- Wzorzec regresji accessibility



https://playwright.dev/docs/aria-snapshots#aria-snapshots

Wywołanie Aria snapshot

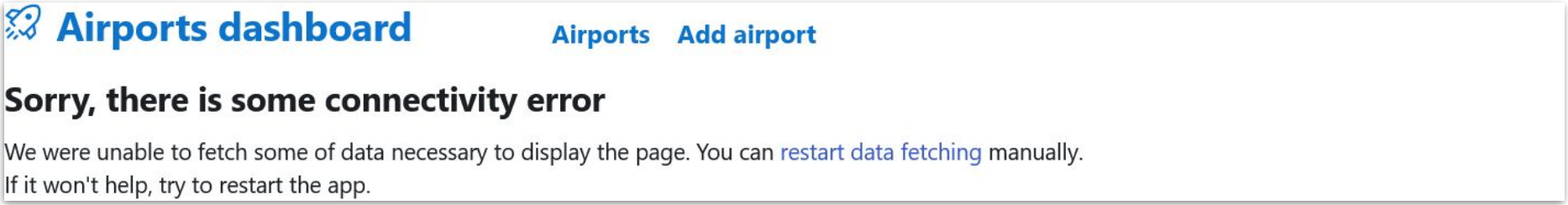
```
private async performAriaSnapshotComparison(snapshotTitle: string) {
    expect(await this.page.locator('body').ariaSnapshot()).toMatchSnapshot([
        ARIA_SNAPSHOTS_DIRECTORY,
        `${this.composeSnapshotName(snapshotTitle)}.yml`,
    ]);
}

private composeSnapshotName(snapshotTitle: string) {
    return slugify(`${this.pageTitle} ${this.testSuite} ${snapshotTitle}`);
}
```

Przykład Aria snapshotu

```
- banner:
 - link "Airports dashboard" :
   - img
   - text: Airports dashboard
 - navigation:
   - list:
     - listitem:
       - link "Airports"
     - listitem:
       - link "Add airport"
- main:
 - alert: Sorry, there is some connectivity error
 - paragraph:
   - text: We were unable to fetch some of data necessary to display the page. You can
   - button "restart data fetching"
   - text: manually.
 - paragraph: If it won't help, try to restart the app.
```

To samo, tylko wizualnie



Nieznana super-moc aria snapshot

```
      - listitem:
        - link "Airports"
      - listitem:
        - link "Add airport"
- main:
  - heading "Add airport" [level=1]
  - text: Name
  - textbox "Name"
  - text: IATA Code
  - textbox "IATA Code"
  - text: Country
  - combobox "Country": Select one
  - text: Regions
```

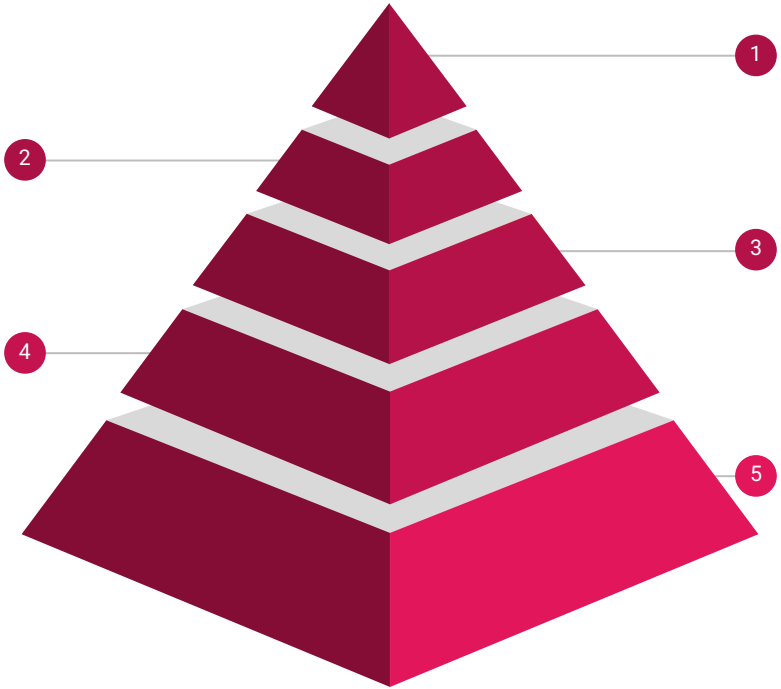
Uwagi i spostrzeżenia

Uwagi i spostrzeżenia

- Testy snapshotów są długie



Czasochłonność testów snapshotów



Testy funkcjonalne dla 60 snapshotów

1

8 sec

Tylko regresja Aria snapshots

2

9 sec

Tylko audyt Axe

3

44 sec

Tylko audyt Lighthouse

114 sec

4

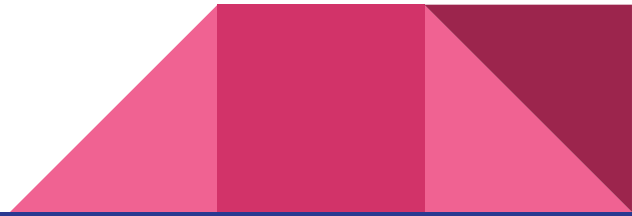
Całościowa snapshot journey

5

132 sec

Uwagi i spostrzeżenia

- Testy snapshotów są długie
- Warto pokrywać a11y na niskim poziomie, np. w design systemie

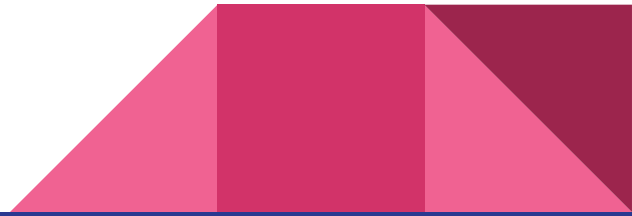




Auditing Design Systems for Accessibility by Anna E. Cook

Uwagi i spostrzeżenia

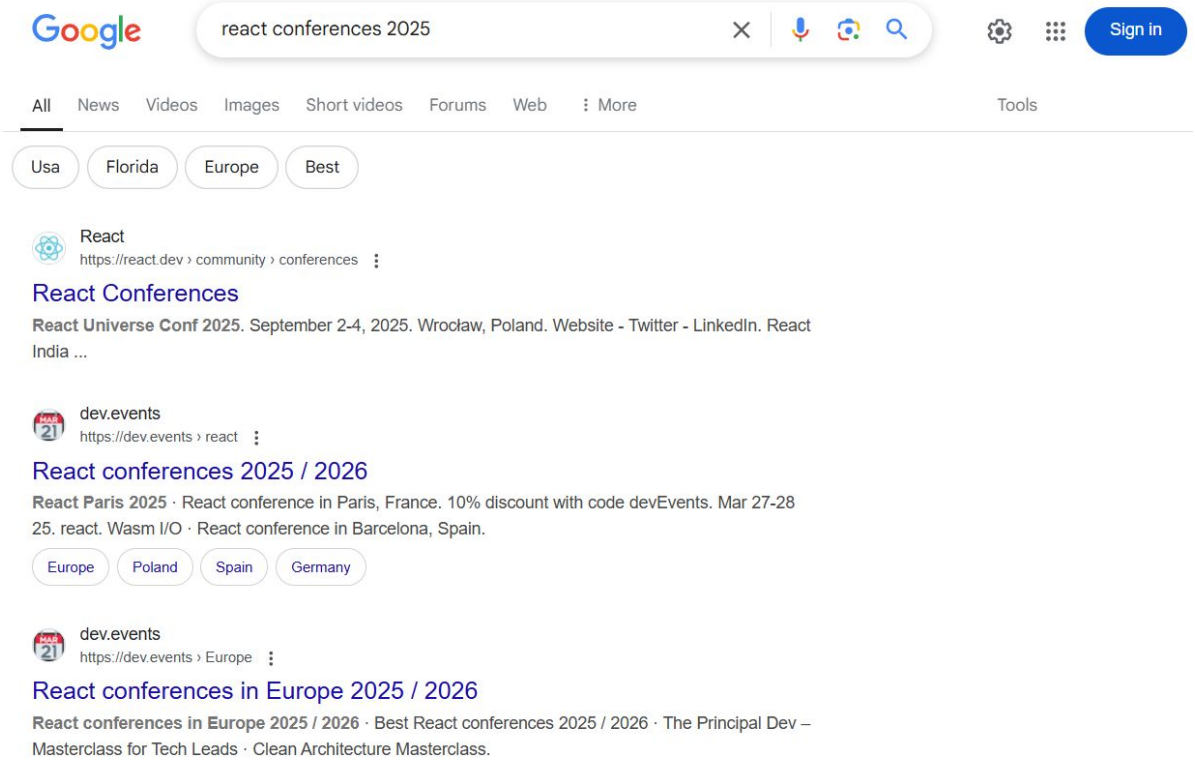
- Testy snapshotów są długie
- Pokrywajcie a11y na niskim poziomie, np. w design systemie
- Ale testujcie też na poziomie aplikacji / produktu



Zakres odpowiedzialności zespołów

Sekcja produktu

Sekcja aplikacji



Uwagi i spostrzeżenia

- Testy snapshotów są długie
- Pokrywajcie a11y na niskim poziomie, np. w design systemie
- Ale testujcie też na poziomie aplikacji / produktu
- Testowanie automatyczne to temat relatywnie młody,
  brakuje mu całościowych rozwiązań, dobrych praktyk i narzędzi
- Uważajcie na opcje płatne



Podsumowanie

Podsumowanie

- A11y is not about being perfect. It’s about trying



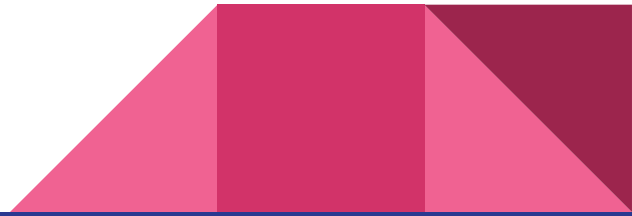
Dodatkowe scenariusze do testów snapshotów





Podsumowanie

- A11y is not about being perfect. It’s about trying
- Pisanie testów pomogło w tworzeniu kultury a11y-aware



Dwa opisy tego samego scenariusza

```
// Given
const search = 'chą szo'

// And text
const fullText = 'Szedł Sasza
    suchą szosą
    i suszył sobie skarpetki'

//
expect(fullText)
    .toContain(search)
```

```
// Given part to be contained in full text
const search = 'search part'

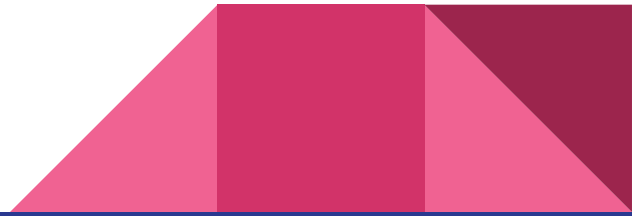
// And text containing searched part
const fullText = `Test ${search} text`



// Then fullText contains search text
expect(fullText)
    .toContain(search)
```

Podsumowanie

- A11y is not about being perfect. It’s about trying
- Pisanie testów pomogło w tworzeniu kultury a11y-aware
- Testy automatyczne nie znajdują wszystkich błędów
- Robią to prawdziwi użytkownicy z assistive technologies



Czego automatyzacja nie wykryje?

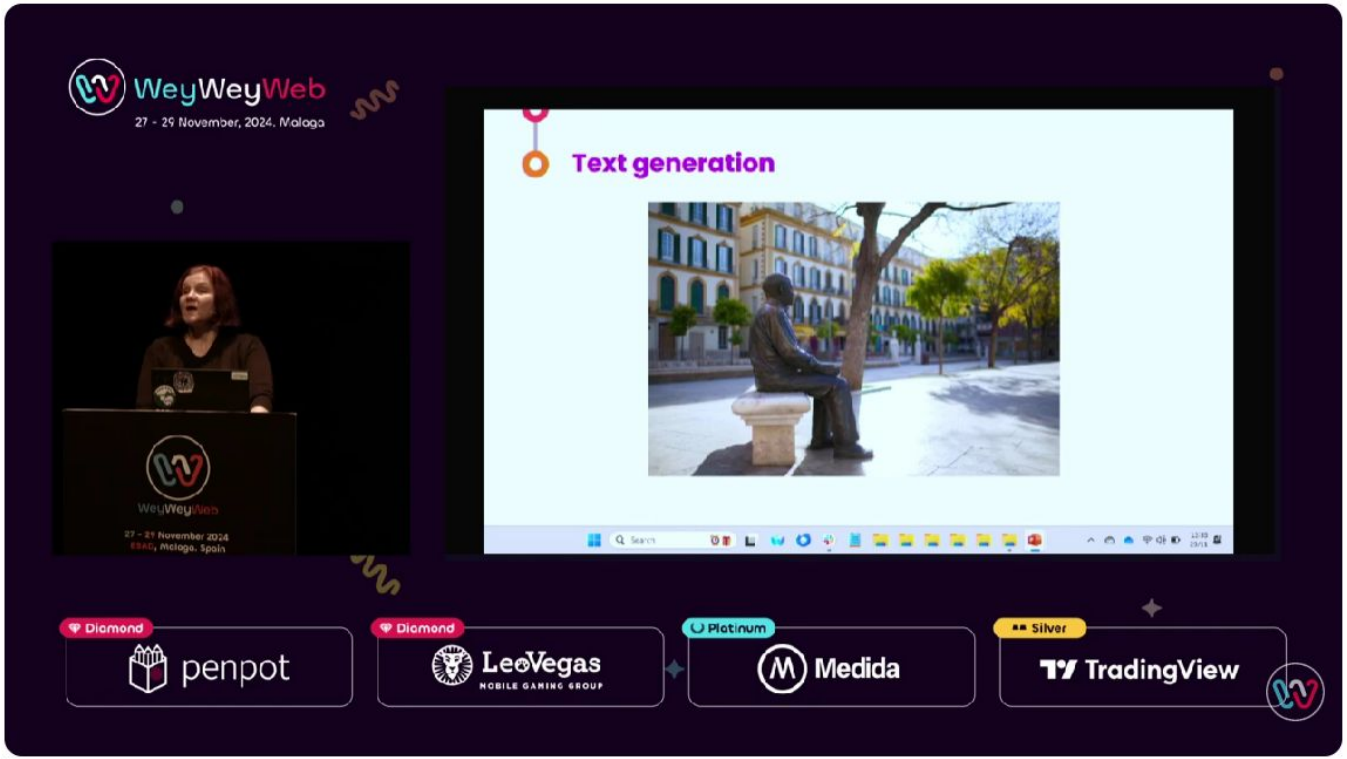
```
<img
  src="picture.png"
  alt="Picture"
/>
```



Co wykryje człowiek?

```
<img
  src="picture.png"
  alt="Obrazek Saszy idącego
suchą szosą, który w trakcie
marszu suszy sobie skarpetki"
/>
```





https://www.youtube.com/watch?v=X8wcbJR9IaU

AI and Accessibility: the Good, the Bad, and the Bollocks by Léonie Watson

Disclaimer od Playwright

DISCLAIMER

Automated accessibility tests can detect some common accessibility problems such as missing or invalid properties. But many accessibility problems can only be discovered through manual testing. We recommend using a combination of automated testing, manual accessibility assessments, and inclusive user testing.

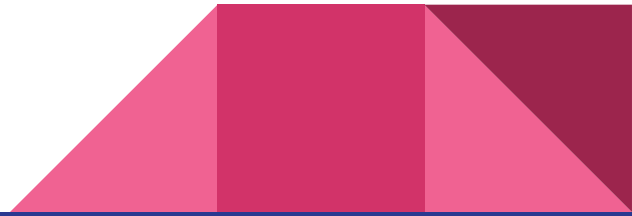
For manual assessments, we recommend Accessibility Insights for Web, a free and open source dev tool that walks you through assessing a website for WCAG 2.1 AA coverage.

Źródło: https://playwright.dev/docs/accessibility-testing

Przykład testu accessibility

Przykład testu accessibility

- Nawigacja z użyciem klawiatury
- Audyt statycznego HTML (testy snapshotów)
  - Analiza z użyciem silnika Axe
  - Audyt Lighthouse
  - Regresja drzewka dostępności



Testy wizualnej regresji

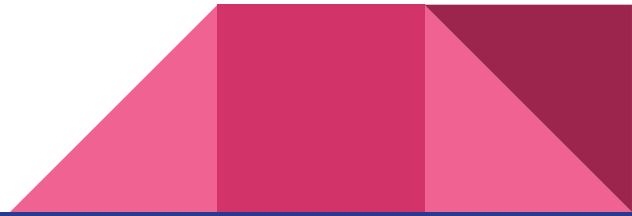
Idea testów wizualnej regresji

- Pojęcie Golden snapshot
- Wykonanie zdjęcia (snapshotu) strony
- Porównanie aktualnego stanu ze złotym snapshotem
  - Brak różnic - jest ok
  - Zgłoszone różnice wymagają sprawdzenia czy one są oczekiwane
- Update złotego snapshotu (jeśli zmiany są oczekiwane)
  - Różnica zostanie zweryfikowana w MR
  - Można wymusić akcept MR przez biznes



Wyzwania związane z automatyzacją

- Obiekty ruchome
- Nieprzewidywalne zachowanie sekcji (np. reklamy, treść hot news, itp.)
- Różnice w malowaniu strony na różnych środowiskach
- Wytrych w postaci maxDiffPixelRatio
- Rozwiązanie z użyciem Dockera



Przykład testu wizualnej regresji

Testy developerskie (integracyjne)

Idea shift left w testowaniu

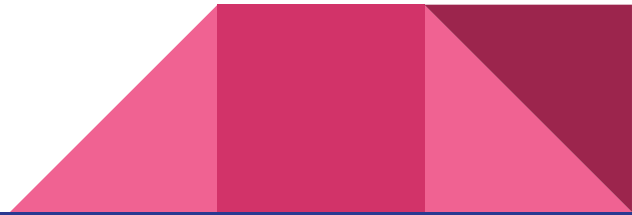
- Testerzy/QA są częścią zespołu developerskiego
  - Włączeni w prace planowania aplikacji/funkcjonalności
  - Weryfikują nowe rozwiązania
  - Automatyzują regresję istniejących rozwiązań
- Developerzy przejmują część obowiązków związanych z automatyzacją testów
  - Piszą dużo testów małej części surowego kodu
- Kultura testowania pozwala na lepsze zaplanowanie aplikacji
  - Od razu myślimy o negative path i edge cases
- Quality people mają przebiegi na rzeczy cool, np. security, wydajność systemu, a11y, eksperymenty




Developerzy
Technologiczna implementacja
Backend
Frontend
db
Testy niskiego poziomu
Testy funkcjonalne
Testy a11y
Testy inne
Testy e2e produktu
Ewangelizacja testowania
Testy security
Vulnerabilities
Nowe rozwiązania
Quality people

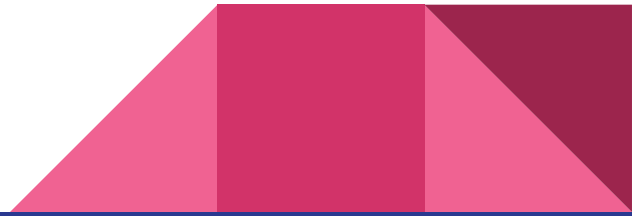
Problemy frontendowców z pisaniem testów

- Opór materii (“Nie jesteśmy od tego”)
- Nacisk na efektywność (wydłuża czas dostarczenia funkcjonalności)
- Brak zrozumienia co należy testować
- Nierzetelna/niejasna wiedza specjalistyczna …
- … co powoduje słabej jakości scenariusze
- … testy dla samego testowania (code coverage)
- … irytację gdy testy są flaky
- … wqrw gdy banalny refactor wywala połowę testów
- … ogólną bardzo słabą jakość w temacie testów



Co zrobić aby pisanie testów było fajne?

- Zrozumieć co powinno być testowane
- Stworzyć stabilne środowisko
  - Vitest + Testing library
- Testować na niskim poziomie
  - Surowy kod komponentów
- Playwright się tutaj nie sprawdzi



Airports dashboard

Airports Add airport

Dark mode

Add airport

Name

Test airport name

IATA Code

TES

Country

Greece

Regions

island

- [ ] Aegean Islands
- [ ] Balearic Islands
- [ ] Canary Islands
- [ ] Corfu Island (Kerkyra)
- [x] Crete Island
- [ ] Dodecanese Islands
- [ ] Ha Long Island
- [ ] Ionian Islands
- [ ] Maiorca Island
- [ ] Rhodes Island
- [ ] Tenerife Island

Vaccination Notes

Submit

Błędy najczęściej popełniane przez developerów

Airports dashboard

Airports Add airport

Dark mode

Add airport

Name

Airport name is required

IATA Code

TES

Country

Greece

Regions

island

- [ ] Aegean Islands
- [ ] Balearic Islands
- [ ] Canary Islands
- [ ] Corfu Island (Kerkyra)
- [x] Crete Island
- [ ] Dodecanese Islands
- [ ] Ha Long Island
- [ ] Ionian Islands
- [ ] Maiorca Island
- [ ] Rhodes Island
- [ ] Tenerife Island

Vaccination Notes

Submit

Test obarczony błędem typu false-positive

```
it('does not display validation error by default' , async () => {
   // When component is rendered
   render(<AddAirportView />)

   // Then the validation message is not displayed
   expect(screen.queryByText('Airport name is required' ))
      .not.toBeInTheDocument ();
});
```



Airports dashboard

Airports Add airport

Dark mode

Propozycja naprawienia testu

```
it('does not display validation error by default' , async () => {
   // When component is rendered
   render(<AddAirportView />)

   // And data loader appeared only during data fetching
   expect(screen.getByTestId('progress-bar')).toBeInTheDocument();
   await waitForElementToBeRemoved(screen.queryByTestId('progress-bar'))

   // Then the validation message is not displayed
   expect(screen.queryByText('Airport name is required'))
      .not.toBeInTheDocument();
});
```

Airports dashboard

Airports Add airport

Dark mode

Sorry, there is some connectivity error

We were unable to fetch some of data necessary to display the page. You can restart data fetching manually.
If it won't help, try to restart the app.

Airports dashboard

Airports Add airport

Dark mode

Add airport

Name

Airport name is required

IATA Code

TES

Country

Greece

Regions

island

- [ ] Aegean Islands
- [ ] Balearic Islands
- [ ] Canary Islands
- [ ] Corfu Island (Kerkyra)
- [x] Crete Island
- [ ] Dodecanese Islands
- [ ] Ha Long Island
- [ ] Ionian Islands
- [ ] Maiorca Island
- [ ] Rhodes Island
- [ ] Tenerife Island

Vaccination Notes

Submit

Testowanie szczegółów implementacyjnych

```
// src/const/routes.ts
const ROUTES = {
   AIRPORTS: '/airports'
} as const
```

```
// tests/AddAirportView.test.tsx
it('redirects to /airports after new airport creation' , async () => {
   const {history} = render(<AddAirportView />)

   // Form fulfillment...

   await userEvent.click(screen.getByRole('button', {name: 'Submit'}))
   expect(history.location.pathname).toEqual(ROUTES.AIRPORTS)
});
```

Co może pójść nie tak?

```
// src/const/routes.ts
const ROUTES = {
   AIRPORTS: '/assports'
} as const
```

```
// tests/AddAirportView.test.tsx
it('redirects to /airports after new airport creation' , async () => {
   const {history} = render(<AddAirportView />)

   // Form fulfillment...

   await userEvent.click(screen.getByRole('button', {name: 'Submit'}))
   expect(history.location.pathname).toEqual(ROUTES.AIRPORTS)
});
```

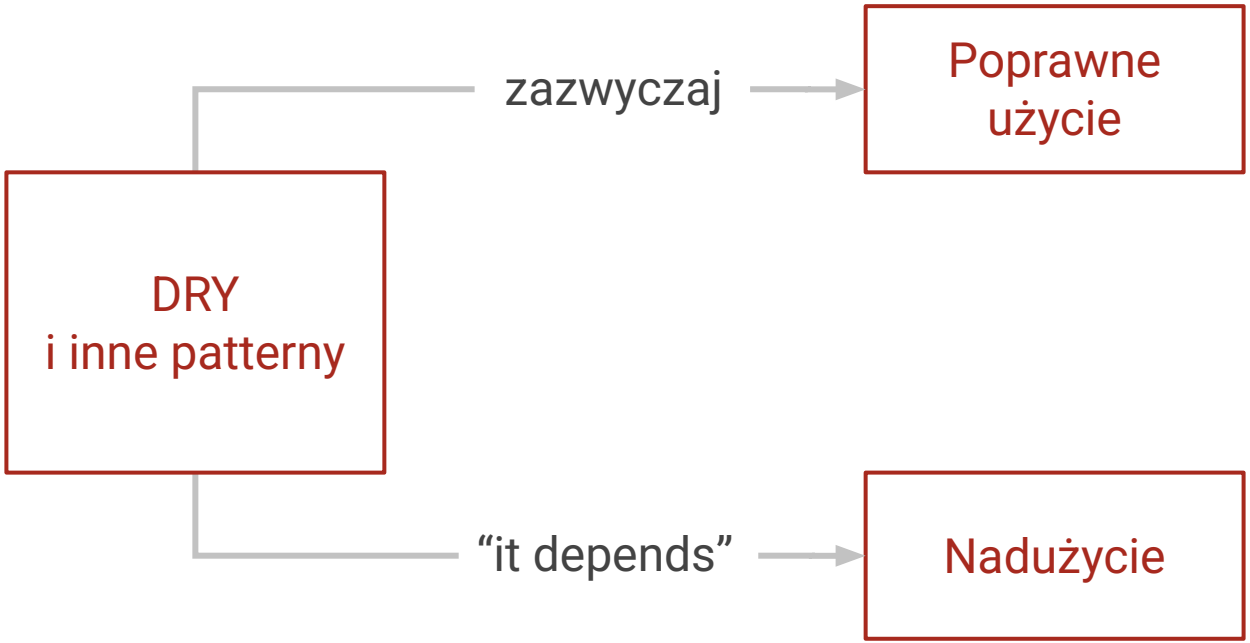
Testowanie funkcjonalności zamiast implementacji

```
// tests/AddAirportView.test.tsx
it('redirects to /airports after new airport creation' , async () => {
   const {history} = render(<AddAirportView />)

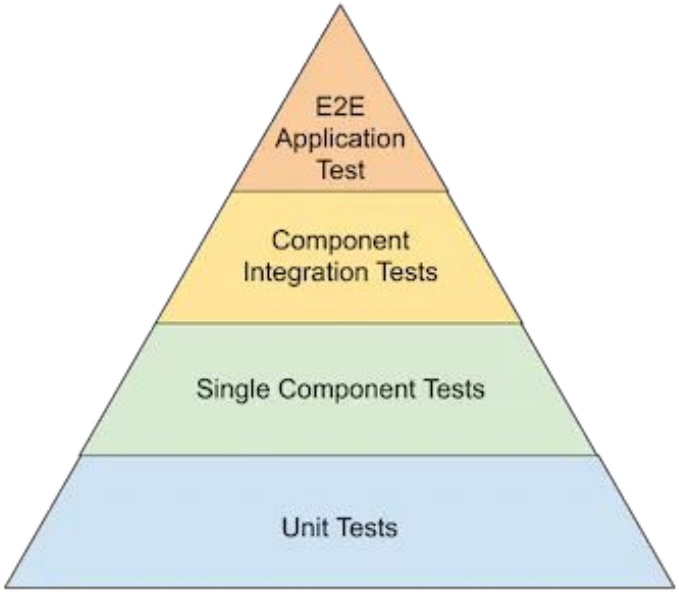
   // Form fulfillment...

   await userEvent.click(screen.getByRole('button', {name: 'Submit'}))
   expect(history.location.pathname).toEqual('/airports')
});
```

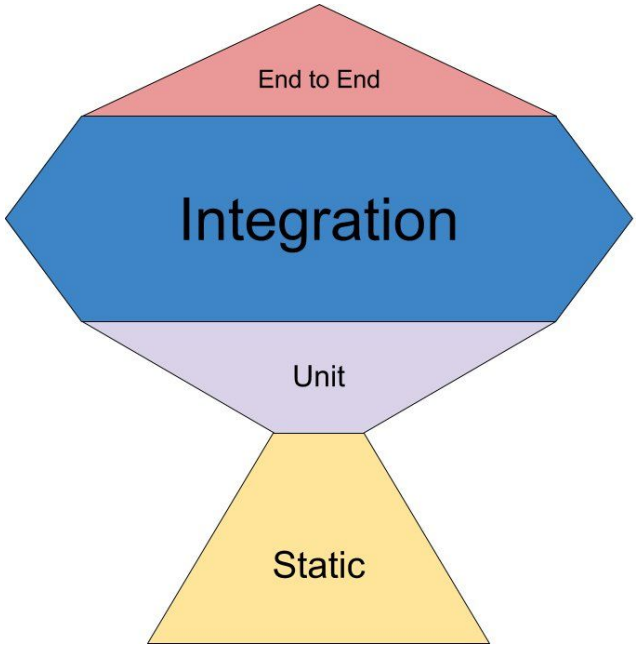
Do czego prowadzą dobre praktyki?



Złe zrozumienie poziomów testowania



Source: https://medium.com/inspiredbrilliance/testing-single-page-applications-b4001adc6452



Source: https://x.com/kentcdodds/status/960723172591992832

”

Testowanie frontendów powinno odzwierciedlać jego ekosystem.

Co mamy bezmyślnie kopiując rozwiązania książkowe?

Source: https://www.youtube.com/watch?v=zE7PKRjrid4

Airports dashboard

Airports Add airport

Dark mode

Add airport

Name

Airport name is required

IATA Code

TES

Country

Greece

Regions

island

- [ ] Aegean Islands
- [ ] Balearic Islands
- [ ] Canary Islands
- [ ] Corfu Island (Kerkyra)
- [x] Crete Island
- [ ] Dodecanese Islands
- [ ] Ha Long Island
- [ ] Ionian Islands
- [ ] Maiorca Island
- [ ] Rhodes Island
- [ ] Tenerife Island

Vaccination Notes

Submit

#1 Testowanie za nisko (ze zbędnym mockowaniem)

```
// tests/AddAirportView/Name.test.tsx
it('displays message when validation return error' , async () => {
    // Given form hook mocked with error being on
    vi.mocked(useField).mockReturnValue({
        meta: {dirty: true, error: 'Test error'}
    } as FieldRenderProps<string>)

    // When component is rendered with error state
    render(<Name />)

    // Then error message is displayed
    expect(screen.getByText('Test error')).toBeInTheDocument()
})
```

#2 Brak pomysłu co należy testować na tym poziomie

```
export const AddAirportView = () => {
    return (<>
        <FormContainer>
            <Name />
            <IataCode />
            <Country />
            <Regions />
            <VaccinationNotes />
        </FormContainer>
    </>)
}
```

#3 Dalszy rozwój aplikacji

```
export const AddAirportView = () => {
    return (<>
        <Collapsible header={'Important!'}>
            <p>Important note content</p>
        </Collapsible>
        <FormContainer>
            <Name />
            <IataCode />
            <Country />
            <Regions />
            <VaccinationNotes />
        </FormContainer>
    </>)
}
```

```
// tests/AddAirportView.test.tsx
it("prints note header and displays content only after click" , async () => {
    // When component render
    render(<AddAirportView />)

    // Then only message header is displayed
    expect(await screen.findByRole('heading', {name: "Important!"}))
        .toBeInTheDocument()
    expect(screen.queryByText('Important note content'))
        .not.toBeInTheDocument()

    // When user click on collapsible element
    await userEvent.click(screen.getByRole('heading', {name: "Important!"}))

    // Then full content is displayed
    expect(await screen.findByText('Important note content'))
        .toBeInTheDocument()
})
```

Pomysł na drobny refaktor

```
export const AddAirportView = () => {
    return (<>
        <ImportantMessage />
        <FormContainer>
            <Name />
            <IataCode />
            <Country />
            <Regions />
            <VaccinationNotes />
        </FormContainer>
    </>)
}
```

#4 Który niszczy architekturę testów

```
export const AddAirportView = () => {
    return (<>
        <ImportantMessage />
        {/* Komponent został przesunięty, ale test jest w tym samym pliku */}
        <FormContainer>
            <Name />
            <IataCode />
            <Country />
            <Regions />
            <VaccinationNotes />
        </FormContainer>
    </>)
}
```

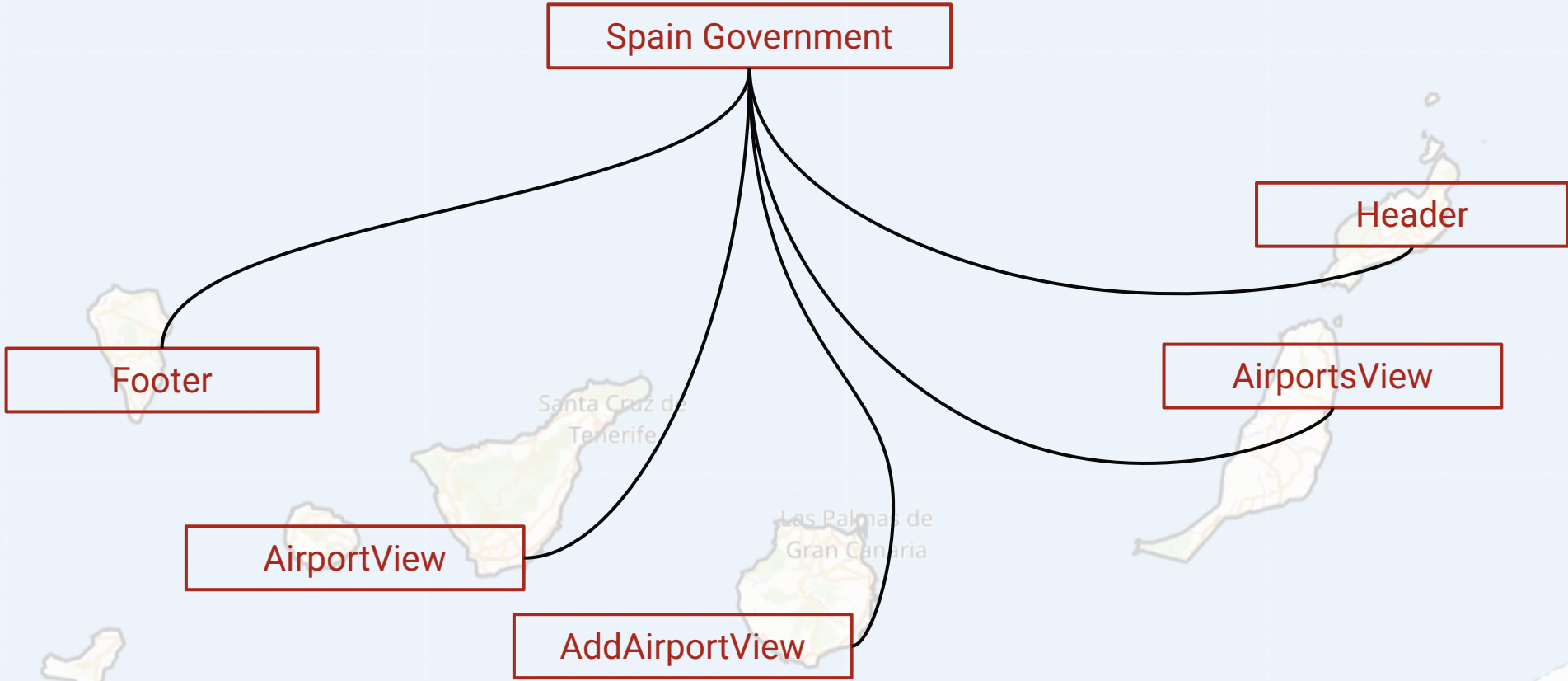
Tak wygląda rzeczywistość testów na froncie

Zacznijmy więc robić to dobrze

Właściwe testowanie

Island architecture pattern




Spain Government
Header
Footer
AirportsView
AirportView
AddAirportView

Island architecture pattern w React Router

```
export const routerDefinition = [
    {
        path: '/', element: <App />, // Lanzarote & La Palma
        children: [
            {
                path: '/airports',
                children: [
                    { index: true, element: <AirportsView />}, // Fuerteventura
                    { path: 'add', element: <AddAirportView />}, // Gran
Canaria
                    { path: ':id', element: <AirportView />}, // Tenerife
                ],
            },
        ],
    },
] satisfies RouteObject[];
```

Nie uciekniemy od zewnętrznych dependencji

```
export const App: React.FC = () =>
    <DesignSystemTheme>
        <AppState>
            <ReactQueryContext>
                <AppLayout>
                    <Outlet />
                { /* ^ either AirportsView, AirportView or AddAirportView */}
                </AppLayout>
            </ReactQueryContext>
        </AppState>
    </DesignSystemTheme>
```

Właściwe testowanie

Wrapowanie komponentów

Implementacja wrappowania komponentów

```
export const renderWithContexts = (component, options = {}) => {
    // Aby dodać React Router
    const {path = '/', url = ''} = options;
    const history = createMemoryHistory({initialEntries: [url]});

    // Aby dodać TanStack Query
    const queryClient = new QueryClient();

    return render(<ReactQueryContext queryClient={queryClient}>
        <HistoryRouter history={history}>
            <Routes>
                <Route path={path} element={component} />
            </Routes>
        </HistoryRouter>
    </ReactQueryContext>)
};
```

Test owrapowanego komponentu

```
✓ test/integration/Views/AddAirportView.test.tsx (1 test)  1405ms
  ✓ AddAirportView (1)
    ✓ Happy path (1)
      ✓ renders component and adds airport  1402ms

Test Files  1 passed (1)
     Tests  1 passed (1)
  Start at  10:10:37
  Duration  5.17s
```

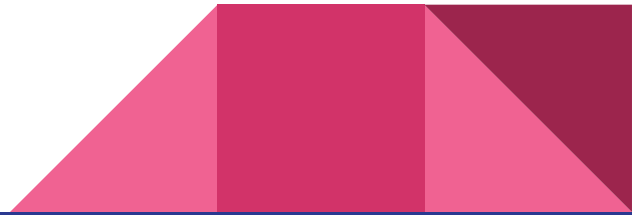
Czy ma to wpływ na performance testu?

```
 RUN  v3.2.4
      Running tests with seed "1757753204226"
      Coverage enabled with istanbul

 ✓ test/integration/test-scenarios/AppFooter.test.tsx  (2 tests) 31ms
   ✓ AppFooter (2)
     ✓ renders raw AppFooter component  11ms
     ✓ renders AppFooter component wrapped in component wrappers  20ms
```

A może wyrenderować całą aplikację?

- Musimy handlować niepotrzebną komunikację z api
- Musimy przenawigować się do testowanego widoku
- Zmagamy się z wyświetlaniem danych związanych z innymi komponentami (wyraźne przy dziedziczonych routach, np. ‘/countries/12/airports/21/airlines’)
- Komplikacja wzrasta przy multistepperach (do testowania kroku n, trzeba przejść przez n-1 wcześniejszych)



Właściwe testowanie

Testy integracyjne

Na czym polegają testy integracyjne?



Kent C. Dodds ⚡
@kentcdodds

The more your tests resemble the way your software is used, the more confidence they can give you.

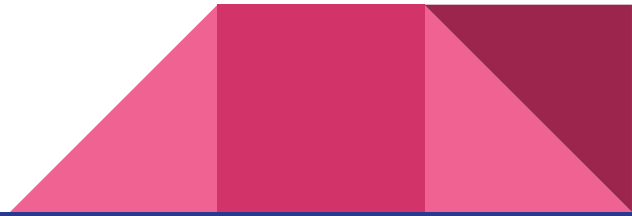
Przetłumacz wpis

4:05 AM · 23 mar 2018

17 340 1 tys. 65

Z czym integrują się frontendy?

- Serwisy backendowe - komunikacja api
  (obsługiwana przez TanStack Query, naśladowana przez MSW)
- Logika wyświetlania widoków - przez url (obsługuje ją React Router)
- Użytkownik - wyświetlenie danych lub integracja z użyciem przycisków, linków czy formularzy (HTML/JS, naśladowanie przez Testing Library family)



Właściwe testowanie

Tworzenie efektywnych scenariuszy

Add airport

Name

Test airport name

IATA Code

TES

Country

Greece

Regions

island

- [ ] Aegean Islands
- [ ] Balearic Islands
- [ ] Canary Islands
- [ ] Corfu Island (Kerkyra)
- [x] Crete Island
- [ ] Dodecanese Islands
- [ ] Ha Long Island
- [ ] Ionian Islands
- [ ] Maiorca Island
- [ ] Rhodes Island
- [ ] Tenerife Island

Vaccination Notes

Submit

Podejście happy i negative path

```
describe('AddAirportsView', () => {
    describe('Happy path', () => {
        // Jakikolwiek scenariusz zakończony funkcjonalnym sukcesem:
        // - wszystkie api odpowiadają 200
        // - kształt wysłanych / odebranych danych się zgadza
        // - wszystko jest wyświetlone i działa jak należy
    }),
    describe('Negative path', () => {
        // Jakikolwiek blocker przed dodaniem lotniska do bazy:
        // - któryś endpoint odpowiedział błędem
        // - użytkownik podał nieprawidłowe dane
    })
});
```

Scenariusze dla AddAirportView

```
describe('AddAirportsView', () => {
    describe('Happy path', () => {
        it('renders view correctly and adds airport' , () => {})
    })
    describe('Negative path', () => {
        it('displays fetch error component on GET api fail' , () => {})
        it('renders validation error' , () => {})
        it('displays error message on POST api fail' , () => {})
    })
})
```

Przykład testu integracyjnego

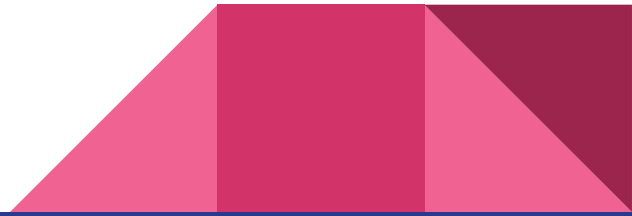
Tips & tricks na poziomie pro+

- Używanie data-test-id to zły pattern
- Dbanie o czystą konsolę
- File parallelism
- Sequence shuffling



Dodatkowe scenariusze testowe

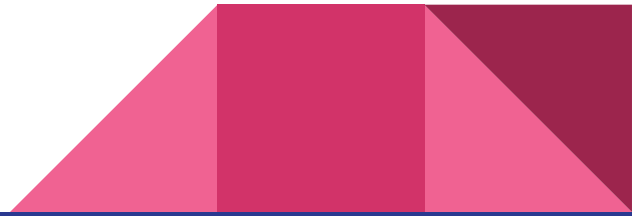
- Testy integracyjne
  - Grupowanie na happy i negative paths, użycie notacji Gherkina, silne mockowanie api
  - wytłumaczenie renderWithContexts() (pattern wrapowania komponentów)
  - Przejście przez scenariusz
- Testowanie State management
  - Po co nam dedykowany dashboard?
  - Konieczność zintegrowania tego z testowanymi widokami
- Testowanie Routingu
  - Pattern z mockowaniem całych widoków
  - Przejście przez test



Podsumujmy warsztaty

Co dziś poznaliśmy?

- Tworzenie kuloodpornych testów funkcjonalnych / e2e w Playwright
- Mockowanie api na poziomie turbo-enterprise
- Testy accessibility
- Testy wizualnej regresji
- Zalety stosowania testing shift left
- Sposoby na wartościowe i solidne testy developerskie



Rekomendacja od developera

- Szlifujcie technologiczną implementację automatyzacji
  - Technologia nie jest Waszą domeną, ale złe jej użycie tworzy testy flaky
- Ewangelizujcie testing shift left wśród swoich managerów i developerów
  - Nie zrzucajcie pracy na developerów. Szukajcie z nimi wspólnych narzędzi do testowania i dzielcie się wiedzą.
- Najlepiej jest zacząć od pisania testów integracyjnych
- Są jak ogarnięcie 80% roboty w 20% czasu, ale tylko jeśli:
  - testują na właściwym poziomie
  - są niezależne od szczegółów implementacyjnych (skupiają się na funkcjonalności)
  - mockują tylko zewnętrzny świat
- Testy funkcjonalne i wizualną regresję implementujcie chwilę przed wydaniem aplikacji
- Automatyczne testy a11y są pomocą a nie rozwiązaniem

Dzięki za uwagę ♡

Q&A

Pamiętajcie o repozytorium z mnóstwem testów!



https://github.com/LukaszNowakPL/bulletproof-frontend-testing

Wartościowe i kuloodporne testy frontendów

Łukasz Nowak, TestWarez2025

Wisła, 05 Listopad 2025